

• Sec-A

1. Write any two difference betⁿ RISC and CISC.

Ans RISC

• it stand for Reduced Instruction Set Computer.

• A very few instruction are present. less than 100.

• RISC does not support an array.

• No condition Codes are used.

• More efficient use of RAM.

CISC

• it stand for Complex Instruction Set Computer.

• A large no. of instructions are present in the architecture.

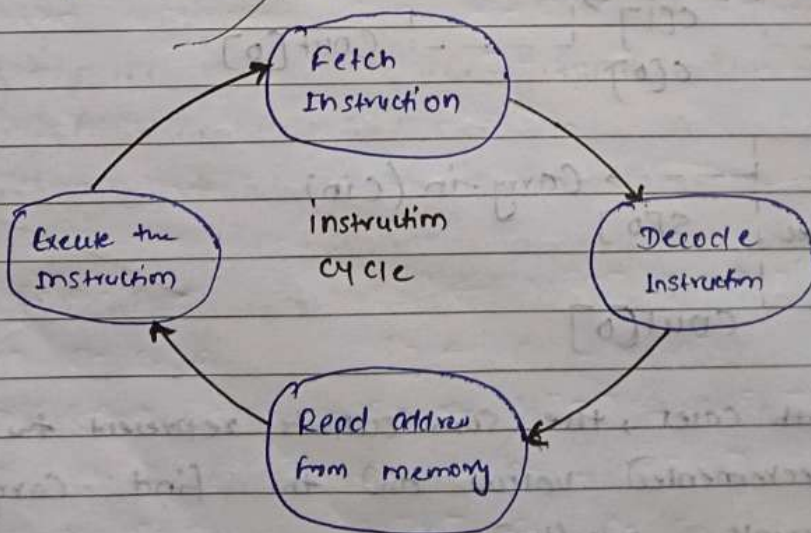
• CISC supports array.

• Condition Codes are used.

• Heavy use of RAM.

2. Define the term instruction cycle with diagram.

Ans instruction cycle is a program residing in the memory unit of a computer consists of a sequence of instructions. These instruction are executed by the processor by going through a cycle for each instruction.



3. What do you mean by Program Counter.

Ans The Program Counter (PC) is a register in a computer processor that indicates where the computer is in its program sequence. it holds the memory address of the

next instruction that the processor fetches in (1111) and execute.

4. What do you mean by register in direct addressing?

Ans A register in direct addressing refers to a special storage location within the CPU that is used to hold such as an address or data value temporarily during the execution of an instruction. Registers in direct addressing are used to store operands or result of operations.

5. How memory differ from buffer.

→ Memory stores data and instructions for processing. A buffer temporarily holds data during transfers between components. Memory is a permanent storage, while a buffer is temporary. Buffer helps smooth out data transfer between components.

• Sec-B ($7 \times 3 = 21$)

1. What do you mean by array multiplier and explain in detail.

→ An array multiplier is a digital combinational circuit used for multiplying two binary numbers by employing an array of full adders and half adders. This array is used for the nearly simultaneous addition of the various product terms involved.

To form various product terms, an array of AND gates is used before the adder array.

The multiplication of two binary numbers can be done with one micro-operation through a combinational circuit that forms the product bits all at once.

Example:

$$\begin{array}{r}
 \begin{array}{cc} b_1 & b_0 \\ a_1 & a_0 \end{array} \\
 \hline
 \begin{array}{cc} a_0 b_1 & a_0 b_0 \\ a_1 b_0 & a_1 b_1 \end{array} \\
 \hline
 \begin{array}{cccc} c_3 & c_2 & c_1 & c_0 \end{array}
 \end{array}$$

Assuming $A = a_1 a_0$ and $B = b_1 b_0$, the various bits of final product term P can be written as

$$p_0 = a_0 b_0$$

$$p_1 = a_1 b_0 + a_0 b_1$$

$$p_2 = a_1 b_1 + c_1$$

$$p_3 = c_2$$

c_1 : The array generated during addition for the p_1 term

c_2 : The array generated during addition for the p_2 term

2. Solve $(A+B \times C) / (D+E-F)$ in 3, 2, 1 and 0 Type of instruction format.

→ Given instruction : $A+B \times C$
 $D+E-F$

(a) Using three (3) instruction format :

$$\text{MUL } R_1, B, C \quad R_1 \leftarrow M[B] * M[C]$$

$$\text{ADD } R_2, R_1, A \quad R_2 \leftarrow R_1 + M[A]$$

$$\text{ADD } R_3, D, E \quad R_3 \leftarrow M[D] + M[E]$$

$$\text{SUB } R_4, R_3, F \quad R_4 \leftarrow R_3 - M[F]$$

$$\text{DIV } X, R_2, R_4 \quad M[X] \leftarrow R_2 / R_4$$

(b) wing two(2) instruction format :

MOV R ₁ , B	$R_1 \leftarrow M[B]$
MUL R ₁ , C	$R_1 \leftarrow R_1 * M[C]$
ADD R ₁ , A	$R_1 \leftarrow R_1 + M[A]$
MOV R ₂ , D	$R_2 \leftarrow M[D]$
ADD R ₂ , E	$R_2 \leftarrow R_2 + M[E]$
SUB R ₂ , F	$R_2 \leftarrow R_2 - M[F]$
DIV R ₁ , R ₂	$R_1 \leftarrow R_1 / R_2$
MOV X, R ₁	$M[X] \leftarrow R_1$

(c) wing one(1) instruction format :

LOAD B	$AC \leftarrow M[B]$
MUL C	$AC \leftarrow AC * M[C]$
ADD A	$AC \leftarrow AC + M[A]$
STORE T	$M[T] \leftarrow AC$
LOAD D	$AC \leftarrow M[D]$
ADD E	$AC \leftarrow AC + M[E]$
SUB F	$AC \leftarrow AC - M[F]$
STORE T ₁	$M[T_1] \leftarrow AC$
LOAD T	$AC \leftarrow M[T]$
DEV T ₁	$AC \leftarrow AC / M[T_1]$
STORE X	$M[X] \leftarrow AC$

(d) wing zero(0) instruction format :

Convert given expression into Prefix :

$ABC * + DE + F - /$

$ABC * + DE + F - /$

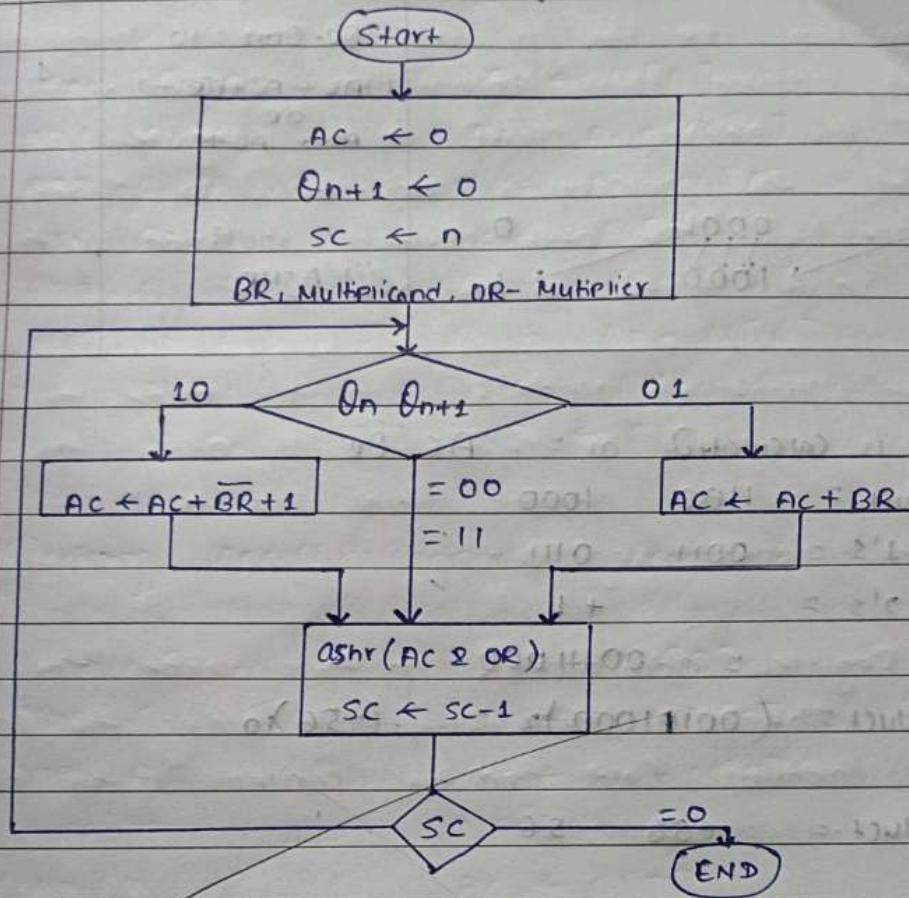
PUSH A	TOS $\leftarrow$ A
PUSH B	TOS $\leftarrow$ B
PUSH C	TOS $\leftarrow$ C
MUL	TOS $\leftarrow$ (B * C)
ADD	TOS $\leftarrow$ (A + (B * C))
PUSH D	TOS $\leftarrow$ D
PUSH E	TOS $\leftarrow$ E
ADD	TOS $\leftarrow$ (D + E)
PUSH F	TOS $\leftarrow$ F
SUB	TOS $\leftarrow$ ((D + E) - F)
DIV	TOS $\leftarrow$ (A + (B * C)) / ((D + E) - F)
POP X	M[X] $\leftarrow$ TOS

3. Design a FlowChart of booth's algorithm.

→ Booth's algorithm is a multiplication algorithm that multiplies two signed binary numbers in two's complement notation. It's particularly efficient for multiplication of large numbers.

Here's a simplified flowChart of

Booth's algorithm :-



• Sec - C

1. Multiply $7 \times (-8)$ using Booth's Algorithm.

→ Here, $M = 7 = (7)_{10} = (0111)_2$ and $\theta = -8 = (1000)_2$

$-M = (-7)_{10} = (1001)_2$

AC	DR	Qn+1	operation	SC
0000	1000	0	Dr. Qn+1 = 00	4
0000	0100	0	(i) ASHR	
0000	0010	0	Dr. Qn+1 = 00	3
0000	0010	0	(i) ASHR	
0000	0001	0	Dr. Qn+1 = 00	2
0000	0001	0	(i) ASHR	

AC	OR	Op+1	Operation	SC
0000 1001			$OR - Op+1 = 10$ (i) $AC \leftarrow AC + OR + 1$ OR $AC \leftarrow AC - M$	1
1001 1100	0001 1000	0 1	(ii) ASHR	

Product is calculated as = AC OR

$$\text{product} = 1100 \quad 1000$$

$$1's = 0011 \quad 0111$$

$$2's = \quad + 1$$

$$= 00111000$$

$$\text{product} = (00101000)_2 = (-56)_{10}$$

$$\therefore \text{product} = \text{E86} - 56$$

2. Explain all types of addressing mode with example.

→ Types of addressing modes :

(i) Implied add. mode: In this, the operands are specified implicitly in the definition of the instruction.

For ex: The instruction "Complement accumulator" is an implied mode instruction because the operand in the accumulator register is implied in the definition of instruction.

(ii) Immediate Mode: it contains a constant value as an operand which is the identification of immediate addressing mode.

Ex: $ADD\ 10\ (AC = AC + 10)$

(iii) Register Mode: in this mode, the instruction specifies a register. This register stores the operand. An example of Register mode: $AC = AC + [R]$

(iv) Register Indirect mode: in this mode, the instruction specifies a register. This register stores the effective address of the operand. An instruction that uses register indirect addressing mode is:
 $AC = AC + [R]$

(v) Direct add. mode: in this mode, the instruction specifies an address. This address is the address of the operand. Example:

$$AC = AC + [X]$$

This will add the operand stored at the address X with the operand stored in the accumulator.

(vi) Indirect add. mode: In this mode, the instruction specifies an address. The memory location specified by the address contains the address of the operand. Example:

$AC = AC + [X]$ This will add the operand stored at the address specified by the memory location X with contents of accumulator.

- (vii) Auto-increment / decrement Mode: In this mode, the instruction specifies a register which points to a memory address that contains the operand. However, after the address stored in the register is accessed, the address is incremented or decremented as specified. The next operand is found by the new value stored in the register.
- (viii) Indexed Addressing Mode: In this mode, the address of the operand is determined by adding the contents of the address field and the contents of the index register.
- (ix) Base Register add. mode: In this mode, the addressing of the operand is determined by adding the contents of the address field and the contents of the base register.
3. Explain the subtraction arithmetic micro-operation.
- Subtraction Arithmetic Micro-operation:

In the subtraction Micro-operation, the contents of register R_2 are subtracted from the contents of the register R_1 and then the result is transferred into R_3 .

$$R_3 \leftarrow R_1 - R_2$$

There is another way of doing the subtraction. In this, 2's complement of R_2 is added to R_1 , which is equivalent to $R_1 - R_2$, and then

the result is transferred into Register R_3 .

$$R_3 \leftarrow R_1 + \bar{R}_2 + 1$$

Here $\bar{R}_2 + 1$, is the 2's complement of R_2 .

We create 1's complement and add 1 to the register which obtains subtracted i.e., $R_1 - R_2$ is similar to $R_3 \leftarrow R_1 + \bar{R}_2 + 1$.

Shirley